

SENSORAGENT

安装部署与使用说明书

面向工业零部件感知、抓取与分拣的具身智能系统



运动控制: HTTP/JSON; 视觉输入: ROS 2 Topic / TF

文档版本	适用平台	发布日期
V1.2 精简版	Ubuntu 22.04 / ROS 2 Humble	2026-09-04

SensorAgent 项目组

挑战杯项目提交文档

导览

本文档用于帮助评委在 Ubuntu 22.04 环境中完成 SensorAgent 的依赖安装、硬件连接和一键运行，并快速了解相机、机械臂、夹爪、ROS 2 与 Agent 之间的通信及系统集成方式。

导览内容	配置说明
应用	根据中文或英文指令识别工业零件，并控制机械臂完成抓取和分拣
仿真	RM65-B + Robotiq 2F-85 + 固定俯视 RGB-D 相机
实机	RM65-B + RealSense D435 + OmniPicker
软件	Ubuntu 22.04、ROS 2 Humble、Python >=3.10

快速运行入口

模式	启动命令
仿真·文本	bash scripts/linux/run_sim_agent.sh
仿真·语音	bash scripts/linux/run_sim_agent.sh --mode voice
实机·文本	bash scripts/linux/run_hardware_agent.sh --enable-motion
实机·语音	bash scripts/linux/run_hardware_agent.sh --enable-motion --mode voice

目录

- 1. 环境配置 4**
 - 1.1 运行环境 4
- 2. 硬件配置 5**
 - 2.1 仿真硬件配置 5
 - 2.2 仿真相机配置 5
 - 2.3 实机硬件清单 5
 - 2.4 实机相机数据 6
 - 2.5 实机机械臂与夹爪参数 6
- 3. 依赖安装 7**
 - 3.1 ROS 2 与系统依赖 7
 - 3.2 准备仿真工作区 7
 - 3.3 准备实机工作区 7
 - 3.4 安装 Python 依赖 7
- 4. 模型与运行配置 8**
 - 4.1 模型文件 8
 - 4.2 双分支配置 8
 - 4.3 启动环境变量 8
- 5. 一键运行完整流程 9**
 - 5.1 运行前检查 9
 - 5.2 仿真链路 9
 - 5.3 实机链路 9
 - 5.4 指令与输出 9
- 6. 通信接口与系统集成 10**
 - 6.1 集成方式 10
 - 6.2 关键节点 10
 - 6.3 HTTP/JSON 接口 10
 - 6.4 ROS 2 接口 11
 - 6.5 完整工作流程 11

1. 环境配置

1.1 运行环境

环境组件	版本 / 配置要求
操作系统	Ubuntu 22.04.5 LTS
ROS	ROS 2 Humble, 系统 Python 3.10
Agent Python	项目支持 >=3.10; 现场使用 3.12.13
CPU / GPU	Intel Core i7-14700HX / RTX 4070 Laptop 8 GB
GPU 软件	NVIDIA Driver 566.36; PyTorch 2.13.0+cu126
主要组件	Transformers 4.57.6、Ultralytics 8.4.117、librealsense 2.58.3

仿真与实机使用同一套 Agent 和任务配置；仿真由 Gazebo Sim、MoveIt 2 和 ros2_control 提供执行环境，实机由 RM65-B、D435 与 OmniPicker 驱动提供执行环境。

2. 硬件配置

2.1 仿真硬件配置

组件	仿真配置
场景	Gazebo Sim; industrial_sorting_metal_pgs.sdf 工业分拣场景
机械臂	RealMan RM65-B, 6 自由度; 实体名 rm65_b_robotiq_2f85
运动规划	MovelIt 2; 规划组 rm_group; base_link -> robotiq_85_tcp
机械臂控制器	rm_group_controller; FollowJointTrajectory; joint1 ~ joint6
夹爪	Robotiq 2F-85; 最大开度 84.8 mm; 最大控制 effort 20.0
夹爪控制	robotiq_gripper_controller/gripper_cmd; 左右 knuckle joint
仿真桥	sensoragent_robot_bridge; http://127.0.0.1:8765

2.2 仿真相机配置

相机参数	仿真配置
相机模型	sensoragent_rgbd_rig; 固定俯视 Gazebo RGB-D 相机
世界坐标	xyz=(0.34, 0, 1.06) m, 朝向工作台
RGB / 深度	424×240, 10 Hz; 水平视场 1.047 rad (约 60°)
裁剪范围	0.10 ~ 3.0 m
ROS 话题	/industrial_camera/image、/industrial_camera/depth_image、 /industrial_camera/camera_info
TF	world -> sensoragent_rgbd_rig/rig/depth_camera

2.3 实机硬件清单

设备	现场配置	连接方式
机械臂	RealMan RM65-B; 6 自由度; 5 kg; 610 mm; 重复定位 ±0.05 mm	Ethernet
相机	Intel RealSense D435; 眼在手安装于 Link6	USB 3.x
夹爪	智元灵犀 X1 OmniPicker; 120 mm 行程; 30 N; 115200 8N1	USB 转 RS485
计算平台	i7-14700HX; RTX 4070 Laptop 8 GB; Ubuntu 22.04	本机

2.4 实机相机数据

相机参数	实机配置
彩色流	1920×1080@30 FPS, BGR8
深度流	1280×720@30 FPS, 深度对齐到彩色图
Agent 发布频率	5 Hz
话题	/vision/raw、/vision/cloud
坐标系	camera_link; 通过眼在手 TF 转换到 base_link

2.5 实机机械臂与夹爪参数

控制参数	实机配置
机械臂地址	192.168.1.19:8080
状态回传	主机 192.168.1.100:8089, UDP 周期 5 ms
夹爪设备	/dev/ttyUSB0, 115200, 8N1
夹爪位置	0.0 为闭合, 1.0 为完全张开
软件工作区	X [-0.55,-0.20] m; Y [-0.26,0.18] m; Z [0,0.35] m

3. 依赖安装

3.1 ROS 2 与系统依赖

按照 ROS 2 官方 Ubuntu 安装流程配置软件源后，安装 Humble、构建工具、RealSense 和运行依赖：

```
sudo apt update
sudo apt install -y ros-humble-desktop ros-humble-rmw-cyclonedds-cpp \
  ros-humble-xacro ros-humble-robot-state-publisher \
  python3-rosdep python3-colcon-common-extensions \
  libturbojpeg0-dev librealsense2-dev librealsense2-utils \
  netcat-openbsd usbutils
rosdep update
```

3.2 准备仿真工作区

首次运行仿真时执行以下准备脚本，自动安装 rosdep 依赖，并构建 RM65-B、Robotiq、Gazebo bringup 与仿真桥：

```
cd ~/SensorAgent
bash scripts/linux/prepare_rm65_b_sim.sh
```

3.3 准备实机工作区

SensorAgent 已包含 RM65-B、OmniPicker、D435 驱动、ROS 2 接口和硬件桥。首次运行实机时执行以下准备脚本，统一安装依赖并完成构建：

```
cd ~/SensorAgent
bash scripts/linux/prepare_hardware_stack.sh
```

3.4 安装 Python 依赖

项目已将核心、语音和视觉依赖统一到一个 requirements.txt：

```
cd ~/SensorAgent
python3.12 -m venv .venv312
source .venv312/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements.txt
python -m pip install onnxruntime
```

4. 模型与运行配置

4.1 模型文件

将项目组提供的模型包放到以下位置。模型权重不通过 Git 下载：

功能	提交后的模型路径
YOLO11 固定类别分支	models/vision/yolo11-seg/industrial-best.pt
Grounding DINO 开放词汇分支	models/vision/grounding-dino/industrial-open-vocab
SAM 2 实例掩码	models/vision/sam2_t.pt
ASR（仅语音模式）	models/asr/sense-voice/model.int8.onnx、tokens.txt
VAD（仅语音模式）	models/asr/vad/silero_vad.onnx

4.2 双分支配置

在 configs/competition_hardware.yaml 中使用现场模型路径：

```
integrations:
  vision:
    backend: dual_branch
    model_path: models/vision/yololl-seg/industrial-best.pt
    grounding_dino_model: models/vision/grounding-dino/industrial-open-vocab
    sam2_model_path: models/vision/sam2_t.pt
    dual_branch:
      route_policy: industrial_first
      fallback_on_error: true
      min_fixed_confidence: 0.35
```

固定类别分支优先识别项目工业类别，例如 roller、hex_nut 和 short_bolt；未知类别、长尾描述、低置信度或固定分支失败时，自动转入 Grounding DINO + SAM 2。仿真 RGB-D 深度或实机 D435 点云用于将二维识别结果转换为 base_link 下的三维坐标。

4.3 启动环境变量

```
export SENSORAGENT_PYTHON=$HOME/SensorAgent/.venv312/bin/python
export SENSORAGENT_ROS_PYTHON=/usr/bin/python3
export SENSORAGENT_ROS_SETUP=/opt/ros/humble/setup.bash
export SENSORAGENT_ROS_WS_SETUP=$HOME/SensorAgent/ros2_ws/install/local_setup.bash
```

如使用默认 LLM 辅助指令理解，再设置 SENSORAGENT_LLM_API_KEY、SENSORAGENT_LLM_MODEL 和 SENSORAGENT_LLM_BASE_URL。不使用联网 LLM 时，在启动命令后增加 --no-llm-grounding。

5. 一键运行完整流程

5.1 运行前检查

检查项	预期结果
机械臂网络	ping 192.168.1.19 成功, TCP 8080 可连接
夹爪串口	/dev/ttyUSB0 存在, 当前用户可读写
D435	rs-enumerate-devices 能找到设备, 使用 USB 3.x
ROS workspace	ros2_ws/install setup 存在, 实机驱动与硬件桥可被 ros2 pkg 找到
视觉模型	第 4.1 节三个视觉模型路径存在
语音模式	ASR、VAD 模型存在, 麦克风可进行 16 kHz 单声道采样
安全	急停有效, 机械臂工作空间内无人且无障碍物

5.2 仿真链路

脚本会启动工业分拣 Gazebo 场景、RM65-B/Robotiq 模型、MoveIt 2、仿真桥和 Agent。仿真默认执行动作；只验证指令解析时增加 --dry-run。

```
# 文本
bash scripts/linux/run_sim_agent.sh

# 语音
bash scripts/linux/run_sim_agent.sh --mode voice
```

5.3 实机链路

统一 launch 同时启动 RM65-B、OmniPicker、D435 驱动节点和 SensorAgent 硬件桥。硬件桥只将 HTTP 请求转换为 ROS 2 Service/Topic 调用，不负责启动驱动或给硬件上电。

```
# 文本
bash scripts/linux/run_hardware_agent.sh --enable-motion

# 语音
bash scripts/linux/run_hardware_agent.sh --enable-motion --mode voice
```

5.4 指令与输出

出现 > 提示后直接输入指令，例如：

```
把左边的螺母放到一号格
把短螺栓放到二号格
把离机械臂最近的滚柱放到三号格
状态
退出
```

每条指令完成后系统返回 IDLE，继续等待下一条。运行结果打印为 JSON，并保存到 logs/tasks/；实机图像和点云保存在 logs/vision/hardware_latest/。

输入 退出 或按 Ctrl-C 正常结束；发生碰撞风险或异常动作时立即使用机械臂物理急停。

6. 通信接口与系统集成

6.1 集成方式

Agent 负责语言理解、双分支视觉、任务编排和恢复；ROS 2 负责相机、TF、运动规划、机械臂及夹爪。运动命令与状态通过本机 HTTP/JSON 桥传递，视觉模块通过下表中的 ROS 2 相机话题和 TF 获取观测。

6.2 关键节点

环境	节点 / 组件	作用
仿真	robot_state_publisher、move_group	发布机器人 TF；完成 RM65-B 运动规划
仿真	controller_manager、rm_group_controller	关节状态与六轴轨迹控制
仿真	robotiq_gripper_action_bridge	将夹爪 Action 转为 Gazebo 力控制命令
仿真	ros_gz_bridge	桥接时钟、RGB、深度与 CameraInfo
仿真	sensoragent_robot_bridge	向 Agent 提供 127.0.0.1:8765 HTTP 接口
实机	rm_driver、arm_control_server	RM65-B 状态通信与运动服务
实机	op_control_node、dep_cam	OmniPicker 控制；D435 图像和点云
实机	sensoragent_hardware_bridge	向 Agent 提供 127.0.0.1:8766 HTTP 接口

6.3 HTTP/JSON 接口

仿真桥地址为 <http://127.0.0.1:8765>，实机桥地址为 <http://127.0.0.1:8766>；两者为 Agent 提供一致的主要控制接口。

接口	用途
GET /health、/ready	检查硬件桥与 ROS 服务状态
GET /state、/gripper/state	读取机械臂和夹爪状态
POST /move-joints、/move-pose、/move-linear	执行机械臂关节、位姿和直线运动
POST /gripper/open、/gripper/close	控制仿真 Robotiq 或实机 OmniPicker
POST /stop	请求停止机械臂运动

6.4 ROS 2 接口

环境 / 类型	接口	用途
仿真 Topic	/industrial_camera/image、/industrial_camera/depth_image、/industrial_camera/camera_info	RGB、深度和内参
仿真 Action	/rm_group_controller/follow_joint_trajectory	RM65-B 六轴轨迹
仿真 Action	/robotiq_gripper_controller/gripper_cmd	Robotiq 夹爪开度
仿真 Topic	/joint_states	机械臂和夹爪关节状态
实机 Topic	/vision/raw、/vision/cloud	D435 彩色图像和对齐点云
实机 Service	/task/arm/movej_deg、/task/arm/move_to_pose、/task/arm/movel	关节、位姿和直线运动
实机 Service	/task/arm/get_current_pose	读取当前末端位姿
实机 Service	/task/op/open、/task/op/close、/task/op/set_position	OmniPicker 开合及指定位置
实机 Topic	/omnipicker_state	夹爪状态反馈
实机 TF	base_link -> Link6 -> camera_link	机械臂与眼在手相机坐标变换

6.5 完整工作流程

自然语言指令 -> 目标与料箱解析
-> ROS 2 相机话题 / TF -> 双分支视觉与三维定位
-> 抓取 / 放置规划 -> HTTP/JSON 仿真桥或实机桥
-> ROS 2 控制接口 -> RM65-B + 夹爪
-> 状态验证 -> 日志记录 -> 返回 IDLE